CSE-1003 • MODULE 2 • CLASS 2

Calculate and decide

Riya's bill learns to do maths and make decisions: totals, averages and a discount.

TODAY'S SYLLABUS TOPICS

Three topics, one smarter bill

1

Operators

Arithmetic, relational, logical

Assignment, increment and decrement

Precedence: which runs first

2

Type conversion and type casting

Implicit conversion: C does it

Explicit casting: you ask for it

The 84.00 average bug, fixed

3

Conditional branching statements

if and if-else

else-if ladder and nested if

switch and break

Last class: Riya's bill printed a total. Today it calculates an average and decides on a discount.

TODAY'S GOAL

A hostel group orders ₹250. The program gives 10% off.

To get there it must **calculate** (operators), get the average **right** (type casting) and **decide** (if-else).

```
How many chai, samosa, Maggi? 10 10 0
Items: 20 Total: Rs 250
Average per item: Rs 12.50
Discount: Rs 25
To pay: Rs 225
```

TOPIC 1 OF 3 • OPERATORS

What is an operator? An action on values

Like the keys on a calculator: + adds, × multiplies. The numbers are what they act on.

```
total = chai * 10 ;
```

- `total` — stores the result
- `=` — assignment operator
- `chai` — operand
- `*` — arithmetic operator
- `10` — operand
- `;` — end

C has five families of operators you'll use all the time:

Arithmetic	Relational	Logical	Assignment	Increment
`+ - * / %`	`> < == !=`	`&& || !`	`= += -=`	`++ --`
calculate	compare: yes or no	combine conditions	store a value	add or take away 1

TOPIC 1 OF 3 • OPERATORS

Arithmetic operators: do the maths

Operator	Meaning	Canteen example	Result
+	Add	20 + 15	35
-	Subtract	100 - 75 (change)	25
*	Multiply	2 * 10 (2 chai)	20
/	Divide	75 / 4 (ints)	18 not 18.75
%	Remainder (modulus)	75 % 10	5

Watch the **/** row: when **both** numbers are int, C drops the decimal part. The next slide shows why.

TOPIC 1 OF 3 • OPERATORS

Integer division and remainder, the samosa way

7 samosas, 2 friends

Friend 1 

Friend 2 

Left over 

/ tells you how many each gets. **%** tells you what is left over.

#	Expression	Result	What happens
1	`7 / 2`	`3`	Each friend gets 3 whole samosas.
2	`7 % 2`	`1`	1 samosa is left over.
3	`7.0 / 2`	`3.5`	One decimal number: the answer keeps .5
4	`10 % 2`	`0`	Nothing left: 10 is even.
5	`47 % 10`	`7`	Remainder by 10 = the last digit.

/ with two ints = whole parts only. % = what's left over.

TOPIC 1 OF 3 • OPERATORS

Relational operators: compare and answer yes or no

Operator	Meaning	Example	Answer
>	greater than	total > 200	1 if yes, 0 if no
<	less than	qty < 5	1 or 0
>=	greater or equal	marks >= 50	1 or 0
<=	less or equal	km <= 2	1 or 0
==	equal to	code == 1	1 or 0
!=	not equal to	qty != 0	1 or 0

= stores a value == compares two values. Mixing them up is the most common bug in if statements.

TOPIC 1 OF 3 • OPERATORS

Logical operators: combine conditions

&&

AND: both must be true

Student **and** total > 100 → 5% off.

isStudent && total > 100

||

OR: at least one true

Tuesday **or** total > 300 → free chai.

day == 2 || total > 300

!

NOT: flips true and false

Canteen is **not** closed → take orders.

!closed

Truth table

A	B	A && B	A||B
1	1	1	1
1	0	0	1
0	1	0	1
0	0	0	0

TOPIC 1 OF 3 • OPERATORS

Assignment operators: keep a running total

```
1  int total = 0;
2  total += 20;
3  total += 15;
4  total += 40;
5  total -= 10;
```

`total += 20`

is short for

`total = total + 20`

Also: -=, *=, /=, %=

#	total	What happens
1	0	Start the bill at 0.
2	20	2 chai: add 20.
3	35	Samosa: add 15.
4	75	Maggi: add 40.
5	65	Coupon: take away 10.

+= adds to what is already there, like a running bill.

TOPIC 1 OF 3 • OPERATORS

Increment and decrement: ++ and --

```
1  int x = 5, y;
2  y = x++;
3  y = ++x;
4  x--;
```

x++: use x, **then** add 1

++x: add 1, **then** use x

Like the canteen token: call the number, then move to the next.

#	x	y	What happens
1	5	?	x starts at 5. y has no value yet.
2	6	5	y = x++: y gets the OLD x (5), then x becomes 6.
3	7	7	y = ++x: x becomes 7 first, then y gets 7.
4	6	7	x--: take 1 away. y doesn't change.

x++ and ++x both add 1. The difference is only what the expression gives back.

TOPIC 1 OF 3 • OPERATORS

Precedence: which operator runs first (C's BODMAS)

1	`( )`	Brackets first
2	`++ -- !`	Increment, decrement, NOT
3	`* / %`	Multiply, divide, remainder
4	`+ -`	Add, subtract
5	`< > ⩽ ⩾`	Compare
6	`== ≠`	Equal, not equal
7	`&& then ||`	AND before OR
8	`= += -=`	Assignment, last

`2 + 3 * 4` → **14**

* first: 3 × 4 = 12, then + 2

`(2 + 3) * 4` → **20**

brackets first: 5 × 4

`2 * 10 + 1 * 15` → **35**

Riya: 20 + 15

`10 / 4 * 2` → **4**

same level: left to right, 2 × 2

Not sure? **Add brackets.** They make the order clear to everyone.

TOPIC 1 OF 3 • OPERATORS • PREDICT

What is the answer?

1 `17 / 5`

3

2 `17 % 5`

2

3 `10 > 5 && 3 > 4`

0: the second part is false

4 `int x = 5; x += 3;`

8

TOPIC 1 OF 3 • OPERATORS • CLASS ACTIVITY

Be the cashier: change, notes and coins

IIIT CANTEEN

Chai × 2	₹20
Samosa × 1	₹15
TOTAL	**₹35**
PAID	**₹100**

The cashier gives change in **₹20 notes** first, then coins.

In pairs • 3 minutes • write C expressions

Q1 Write the change as one C expression.

100 - (2 * 10 + 15)
= 65

Q2 How many ₹20 notes? Which operator?

65 / 20 = 3 notes

Q3 How many rupees in coins?

65 % 20 = ₹5

Q4 Remove the brackets from Q1. What do you get?

100 - 2*10 + 15 = 95 ×

TOPIC 1 OF 3 • OPERATORS • CLASS ACTIVITY

Who gets the offer? You decide

Offer 1: 5% off

```
isStudent && total > 100
```

Offer 2: free chai

```
day == TUESDAY || total > 300
```



Riya

Student	**Yes**
Total	**₹75**
Day	**Monday**

Offer 1: ✕ 75 is not > 100
Offer 2: ✕ neither part



Arjun

Student	**Yes**
Total	**₹150**
Day	**Wednesday**

Offer 1: ✓ both true
Offer 2: ✕ neither part



Meera

Student	**No**
Total	**₹320**
Day	**Friday**

Offer 1: ✕ not a student
Offer 2: ✓ 320 > 300



Kabir

Student	**No**
Total	**₹90**
Day	**Tuesday**

Offer 1: ✕ neither part
Offer 2: ✓ it is Tuesday

TOPIC 1 OF 3 • OPERATORS • FILL IN THE BLANKS

Operators practice: 7 blanks, 7 operators

```
/* Operators: fill in the 7 blanks marked ___ */
#include <stdio.h>

int main(void) {
    int chai = 2, samosa = 1, paid = 100;
    int total, change, notes, coins;
    int isStudent = 1, token = 0;

    total = chai * 10 ___ samosa * 15; /* 1 */
    change = paid ___ total; /* 2 */
    notes = change ___ 20; /* 3 */
    coins = change ___ 20; /* 4 */
    printf("Change %d: %d notes, Rs %d\n", change, notes, coins);

    printf("Offer? %d\n", isStudent ___ total > 30); /* 5 */
    token___; /* 6 */
    total ___ 5; /* 7 */
    printf("Token %d, new total %d\n", token, total);
    return 0;
}
```

The cashier slide as code. Pick the operator for each blank, then run it.

Correct output

```
Change 65: 3 notes, Rs 5
Offer? 1
Token 1, new total 30
```

Scan or open:

onlinegdb.com/v5nx6m5Pa

In pairs • 5 minutes

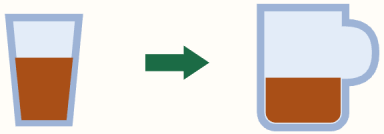
TOPIC 1 OF 3 • OPERATORS • ANSWERS

Check your operators

1 `1: total = chai * 10 ___ samosa * 15;`	+
2 `2: change = paid ___ total;`	-
3 `3: notes = change ___ 20;`	/ (whole notes)
4 `4: coins = change ___ 20;`	% (what is left)
5 `5: isStudent ___ total > 30`	&&
6 `6: token___;`	++
7 `7: total ___ 5;`	-= (coupon)

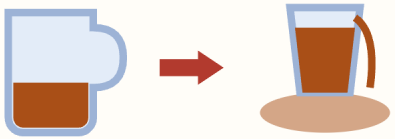
TOPIC 2 OF 3 • TYPE CONVERSION AND TYPE CASTING

What is type conversion? Moving a value to a different type



Glass → jug: safe

Pour chai from a glass into a big jug: nothing is lost. int 7 → float 7.0



Jug → glass: you lose some

Pour a full jug into a small glass: it spills. float 7.9 → int 7

Implicit conversion

C converts **automatically** when you mix types.

```
float price = 7; // 7 becomes 7.0
```

Explicit casting

You ask for a conversion with (type).

```
avg = (float) total / items;
```

Implicit conversion: what C does on its own

Expression	Rule	Result
5 + 2.5	int + float → float	7.5
7 / 2	int / int → int	3 (decimal dropped)
7 / 2.0	int / float → float	3.5
int n = 9.99;	float stored in int	9 (cut, not rounded)
'A' + 1	char + int → int	66 (ASCII of A is 65)

The rule: in a mixed calculation, C converts to the **bigger** type first. Storing into a smaller type **cuts** the extra part.

TOPIC 2 OF 3 • TYPE CONVERSION AND TYPE CASTING

The 84.00 bug, step by step

```
1  int a = 85, b = 90, c = 78;
2  float avg;
3  avg = (a + b + c) / 3;
4  avg = (float)(a + b + c) / 3;
```

The average-of-marks bug from our first class. Now we can explain it, and fix it.

#	Value	Type	What happens
1	`a + b + c`	`253 int`	All three are int, so the sum is an int.
2	`253 / 3`	`84 int`	int ÷ int: the .33 is thrown away here.
3	`avg`	`84.00`	Stored in a float, but too late. The decimals are already gone.
4	`(float)253`	`253.0`	Fix: cast the sum to float first.
5	`253.0 / 3`	`84.33`	float ÷ int → float. The decimals stay.
6	`avg`	`84.33`	Correct!

Convert BEFORE you divide, not after.

Explicit casting: (type) value

`(float) total / items` means: treat total as a float for this calculation.

Expression	What happens	Result
(float) 75 / 4	75.0 / 4	18.75
(float)(75 / 4)	(float) 18	18.00
(int) 9.99	cut the decimals	9
(int) 'A'	character to its code	65

Where the cast goes matters. Rows 1 and 2 look alike, but row 2 divides first, so the decimals are already lost.

TOPIC 2 OF 3 • TYPE CONVERSION AND TYPE CASTING • PREDICT

What is the result?

1	`7 / 2`	3
2	`7 / 2.0`	3.5
3	`(float) 7 / 2`	3.5
4	`(float)(7 / 2)`	3.0: divided first

TOPIC 2 OF 3 • TYPE CONVERSION AND TYPE CASTING • CLASS ACTIVITY

Split the bill: is it fair?



Three friends share a **₹250** bill. The app works out each share.

Riya's CGPA is **8.75**. The app stores it.

For each line of code, predict the answer. Is it fair, or did someone lose money or marks?

In pairs • 3 minutes

Q 1

```
int share = 250 / 3;
```

83: ₹1 goes missing

Q 2

```
float share = (float) 250 / 3;
```

83.33: fair

Q 3

```
int cgpa = 8.75;
```

8: Riya loses 0.75!

Q 4

```
float share = (float)(250 / 3);
```

83.00: cast too late

TOPIC 2 OF 3 • TYPE CONVERSION AND TYPE CASTING • FILL IN THE BLANKS

Type conversion practice: 5 blanks

```c
/* Type conversion: fill in the 5 blanks marked ___ */
#include <stdio.h>

int main(void) {
    int bill = 250, friends = 3, stars, code;
    float wrong, right, half;

    wrong = bill / friends;
    right = (_____) bill / friends; /* 1 */
    stars = (___) 4.6; /* 2 */
    code = ___; /* 3: the letter B */
    half = 15 / ___; /* 4 */

    printf("Wrong share: %.2f\n", wrong);
    printf("Right share: %.2f\n", right);
    printf("Stars: %d\n", stars);
    printf("Code of B: %d\n", code);
    printf("Half samosa: Rs %____\n", half); /* 5 */
    return 0;
}
```

Make every line print the **right** value, then run it.

Correct output

```
Wrong share: 83.00
Right share: 83.33
Stars: 4
Code of B: 66
Half samosa: Rs 7.50
```

Scan or open:

onlinegdb.com/QD8AGNrgP

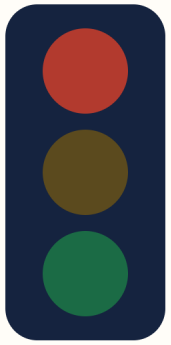
In pairs • 5 minutes

Check your answers

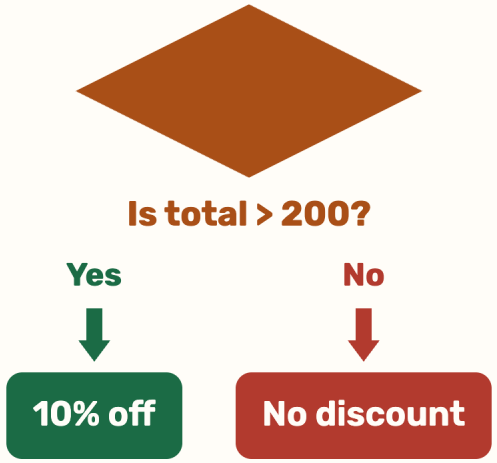
1	`1: right = (_____) bill / friends;`	float
2	`2: stars = (___) 4.6;`	int (gives 4, not 5)
3	`3: code = ___; (the letter B)`	'B' (stored as 66)
4	`4: half = 15 / ___;`	2.0 (int ÷ float = float)
5	`5: printf("... Rs %____\n", half);`	.2f

TOPIC 3 OF 3 • CONDITIONAL BRANCHING STATEMENTS

What is a conditional statement? A program that chooses



A traffic signal: **if** red, stop; **else** go. You check, then choose.



Five ways to decide in C

`if`	do it only when true
`if-else`	choose one of two paths
`else-if ladder`	choose one of many
`nested if`	a decision inside a decision
`switch`	match one value against cases

TOPIC 3 OF 3 • CONDITIONAL BRANCHING STATEMENTS

if: do something only when the condition is true

```
if (total > 200) {
    printf("You get 10% off!\n");
}
printf("Thank you!\n");
```

1. The condition goes in ( ), using relational and logical operators.
2. True (1): the block in { } runs.
3. False (0): the block is skipped.
4. Line 4 runs either way: it is outside the braces.

TOPIC 3 OF 3 • CONDITIONAL BRANCHING STATEMENTS

if-else: choose one of two paths

```
1  if (total > 200) {
2      discount = total / 10;
3  } else {
4      discount = 0;
5  }
```

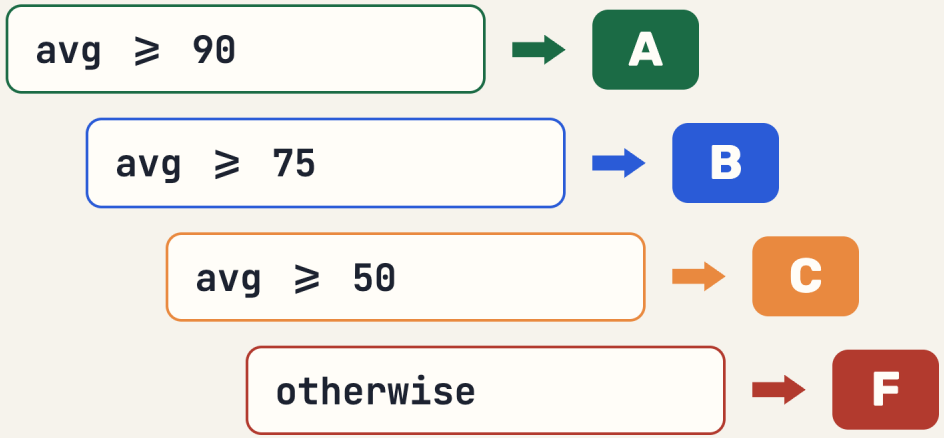
Exactly **one** of the two blocks runs, never both, never neither.

#	total	total > 200?	discount	What happens
1	75	No (0)	0	Riya: the else block runs.
2	250	Yes (1)	25	Hostel group: the if block runs, 250 / 10.
3	200	No (0)	0	Exactly 200 is NOT more than 200.

Test the boundary: 200 gets no discount because > means 'more than'.

TOPIC 3 OF 3 • CONDITIONAL BRANCHING STATEMENTS

else-if ladder: choose one of many

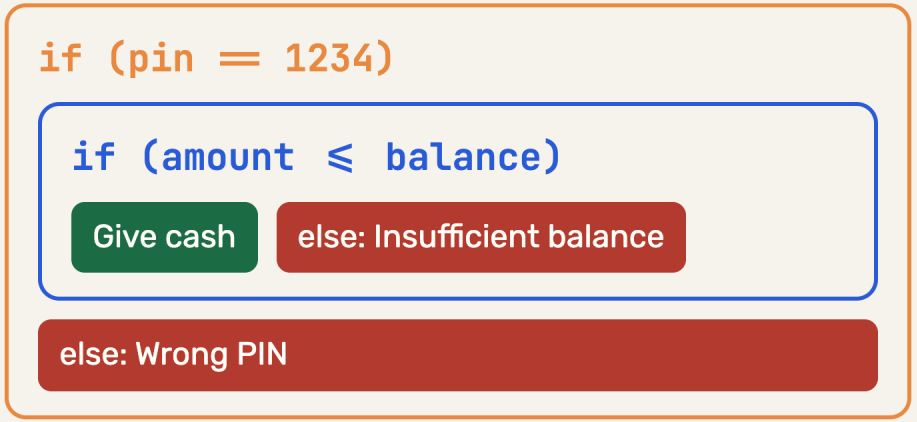


C checks from the **top** and stops at the **first** true condition. An average of 81 skips A, matches B, and never checks C.

```
if (avg ⩾ 90) {
    printf("A");
} else if (avg ⩾ 75) {
    printf("B");
} else if (avg ⩾ 50) {
    printf("C");
} else {
    printf("F");
}
```

TOPIC 3 OF 3 • CONDITIONAL BRANCHING STATEMENTS

Nested if: a decision inside a decision




```
if (pin == 1234) {
    if (amount ⩽ balance) {
        printf("Take your cash");
    } else {
        printf("Insufficient balance");
    }
} else {
    printf("Wrong PIN");
}
```

The ATM checks the balance **only if** the PIN is right. The inner decision waits for the outer one.

TOPIC 3 OF 3 • CONDITIONAL BRANCHING STATEMENTS

switch: like a canteen machine with numbered buttons

CANTEEN MACHINE

Enter code: 2

1 **Chai** ₹10

2 **Samosa** ₹15

3 **Maggi** ₹40

Any other code → "No such item"

```
switch (code) {
    case 1: price = 10; break;
    case 2: price = 15; break;
    case 3: price = 40; break;
    default: price = 0;
             printf("No such item");
}
```

`switch (code)`
Press a button: the value to check

`case 2:`
Go straight to slot 2

`break;`
Item drops: stop here

`default:`
Any other button: error message

switch or if-else? switch for **exact values** of one variable: menu codes, days, grades A/B/C. if-else for **ranges**: marks >= 75, total > 200.

TOPIC 3 OF 3 • CONDITIONAL BRANCHING STATEMENTS

What happens without break? Fall-through

Like a lift with broken stop buttons: press 2 and the doors open at 2, **and** at every floor after it.

```
switch (code) {
    case 1: price = 10;
    case 2: price = 15;
    case 3: price = 40;
    default: price = 0;
}
```

Riya presses **code = 2** (samosa). The breaks are missing.

#	Line	price	What happens
1	2	–	code is 2: case 1 is skipped.
2	3	15	Jump to case 2: price = 15.
3	4	40	No break! Falls into case 3: price = 40.
4	5	0	Falls into default: price = 0.

Samosa costs Rs 0? Every case needs a break.

Fixed with break, and when fall-through helps

Without break ✕: samosa × 3 costs Rs 0

```
switch (code) {
    case 1: price = 10;
    case 2: price = 15;
    case 3: price = 40;
    default: price = 0;
}
```

```
Item code and quantity? 2 3
Amount: Rs 0
```

With break ✓: stops at case 2

```
switch (code) {
    case 1: price = 10; break;
    case 2: price = 15; break;
    case 3: price = 40; break;
    default: price = 0;
}
```

```
Item code and quantity? 2 3
Amount: Rs 45
```

Useful fall-through

Leave out break **on purpose** to give cases the same action. Day 6 falls into day 7.

```
case 6:
case 7: printf("Weekend: opens at 9"); break;
default: printf("Weekday: opens at 8");
```

TOPIC 3 OF 3 • CONDITIONAL BRANCHING STATEMENTS • PREDICT

What gets printed?

1

```
int t = 200; if (t > 200) printf("Off");
else printf("Full");
```

Full

2

```
float avg = 75; if (avg ⩾ 90) printf("A"); else if (avg
⩾ 75) printf("B"); else printf("C");
```

B: 75 >= 90 is false, 75 >= 75 is true

3

```
x = 3; switch (x) { case 3: printf("Maggi"); break;
default: printf("?"); }
```

Maggi

ALL 3 TOPICS TOGETHER

Riya's bill, now with average and discount

13–15	**1 • Arithmetic: + and *, with precedence**
16	**2 • Casting: (float) before /**
18	**1 • Relational: total > 200**
18–22	**3 • if-else: discount or not**
19	**1 • / gives 10%**
25	**%.2f prints the average**

Lines 1 to 11 are the same ideas as Class 1.

```c
#include <stdio.h>

int main(void) {
    const int CHAI_PRICE = 10;
    const int SAMOSA_PRICE = 15;
    const int MAGGI_PRICE = 40;
    int chai, samosa, maggi, items, total, discount;
    float avg;

    printf("How many chai, samosa, Maggi? ");
    scanf("%d %d %d", &chai, &samosa, &maggi);

    items = chai + samosa + maggi;
    total = chai * CHAI_PRICE + samosa * SAMOSA_PRICE
          + maggi * MAGGI_PRICE;
    avg = (float) total / items;

    if (total > 200) {
        discount = total / 10;
    } else {
        discount = 0;
    }

    printf("Items: %d Total: Rs %d\n", items, total);
    printf("Average per item: Rs %.2f\n", avg);
    printf("Discount: Rs %d\n", discount);
    printf("To pay: Rs %d\n", total - discount);
    return 0;
}
```

PREDICT BEFORE YOU RUN

What will the program print?

1	Input: 2 1 1. Average? To pay?	18.75 and Rs 75
2	Input: 10 10 0. Discount? To pay?	Rs 25 and Rs 225
3	Input: 0 0 5 (Rs 200). Discount?	Rs 0: 200 is not > 200

RUN IT YOURSELF

Two programs to try in OnlineGDB

bill2.c

The canteen bill with average and discount.

Type in the input box first: **10 10 0**

Scan or open:

onlinegdb.com/rUwfGyteW

menu.c

A switch menu: item code and quantity.

Type in the input box first: **2 3**

Scan or open:

onlinegdb.com/vytRSpLVMZ

YOUR TURN • FILL IN THE BLANKS

Complete the program: 5 blanks

```c
/* Fill in the 5 blanks marked ____ */
#include <stdio.h>

int main(void) {
    int items, total, discount;
    float avg;

    printf("Items and total? ");
    scanf("%d %d", &items, &total);

    avg = (_____) total / items; /* blank 1 */

    if (total ___ 200) { /* blank 2 */
        discount = total __ 10; /* blank 3 */
    } ____ { /* blank 4 */
        discount = 0;
    }

    printf("Average: Rs %___\n", avg); /* blank 5 */
    printf("To pay: Rs %d\n", total - discount);
    return 0;
}
```

One blank per idea from today: a cast, a comparison, an operator, an else and a format.

Scan or open:

onlinegdb.com/VRSF3kAW4

Work in pairs • 5 minutes

FILL IN THE BLANKS • ANSWERS

Check your answers

1	`Blank 1: avg = (_____) total / items;`	**float**
2	`Blank 2: if (total ___ 200)`	**>**
3	`Blank 3: discount = total __ 10;`	**/**
4	`Blank 4: } ____ {`	**else**
5	`Blank 5: printf("Average: Rs %___\n", avg);`	**.2f**

YOUR TURN • TASK 1 OF 3

Electricity bill with slabs



The first 100 units cost Rs 5 each. Every unit above 100 costs Rs 7. Print the bill.

if-else **boundary: 100**

Pseudocode

```
READ units
IF units ⩽ 100 THEN
    bill ← units × 5
ELSE
    bill ← 500 + (units - 100) × 7
END IF
PRINT bill
```

Sample runs

```
Units used? 80
Bill = Rs 400

Units used? 150
Bill = Rs 850
```

Check yourself

- ✓ Which operator means 'up to 100'?
- ✓ Test exactly 100: it should be Rs 500
- ✓ Use braces for each block

Now: Write it in C in OnlineGDB. Test every sample, including the boundary.

YOUR TURN • TASK 2 OF 3

Grade calculator



Read marks in 3 subjects. Find the average and print the grade: A (90+), B (75+), C (50+), else F.

casting **else-if ladder**

Pseudocode

```
READ m1, m2, m3
avg ← (float)(m1 + m2 + m3) / 3
IF avg ⩾ 90 THEN grade ← A
ELSE IF avg ⩾ 75 THEN grade ← B
ELSE IF avg ⩾ 50 THEN grade ← C
ELSE grade ← F
PRINT avg, grade
```

Sample runs

```
Marks in 3 subjects? 85
90 70
Average: 81.67 Grade: B
```

Check yourself

- ✓ Cast BEFORE you divide
- ✓ Check the ladder from the top: 90 first
- ✓ Test 90, 90, 90: it should be A

Now: Write it in C in OnlineGDB. Test every sample, including the boundary.

YOUR TURN • TASK 3 OF 3

Auto fare, now in C



An auto charges Rs 30 for the first 2 km and Rs 15 per km after that. You wrote this as pseudocode in Module 1. Now write it in C.

if-else **boundary: 2 km**

Pseudocode

```
READ km
IF km ⩽ 2 THEN
    fare ← 30
ELSE
    fare ← 30 + (km - 2) × 15
END IF
PRINT fare
```

Sample runs

```
Distance in km? 1
Fare = Rs 30
Distance in km? 2
Fare = Rs 30
Distance in km? 10
Fare = Rs 150
```

Check yourself

- ✓ Same steps as your Module 1 answer
- ✓ Test 1, 2 and 10 km
- ✓ 10 km: 30 + 8 × 15 = 150

Now: Write it in C in OnlineGDB. Test every sample, including the boundary.

TODAY IN ONE SLIDE

Calculate, convert, decide

1

Operators

Arithmetic + − * / %, relational > < == !=, logical && || !, assignment = +=, increment ++ --. Brackets settle the order.

2

Type conversion

C converts to the bigger type in mixed maths. Storing in a smaller type cuts decimals. Cast with (float) **before** dividing.

3

Decisions

if, if-else, else-if ladder, nested if, switch with break. Always test the boundary value.

NEXT CLASS

Riya's friends keep ordering until they say stop.
How does a program repeat?

Class 3: while, do-while, for, nested loops, break and continue.

Before next class: finish one task and fix the broken program.

YOUR TURN • FIX THE BROKEN CODE

This discount program has 3 bugs

```
 1  /* Riya's bill with discount: find and fix 3
bugs */
 2  #include <stdio.h>
 3
 4  int main(void) {
 5      int items, total, discount = 0;
 6      float avg;
 7
 8      printf("Items and total? ");
 9      scanf("%d %d", &items, &total);
10
11      avg = total / items;
12
13      if (total > 200);
14          discount = total * 10;
15
16      printf("Average: Rs %.2f\n", avg);
17      printf("To pay: Rs %d\n", total
- discount);
18      return 0;
```

One bug from each topic today. Run it with input **4 250** and look at the answer!

Line 11: total / items is int ÷ int

2 • Cast: (float) total / items

Line 13: the ; ends the if

3 • Discount is always applied

Line 14: * 10 should be / 10

1 • Wrong operator

[QR code and OnlineGDB link: broken_bill2.c]

With 4 250 it prints **Average: 62.00** and **To pay: Rs -2250**. A program can run without errors and still be badly wrong.